



Estructuras de Datos

Grado en Ingeniería en Informática

Examen de la convocatoria extraordinaria I 2015/2016

Nombre:

Grupo:

Pregunta 1:

Contestar V/F las siguientes cuestiones, teniendo en cuenta que una respuesta incorrecta invalida una correcta.

| | Tanto si una matriz de 100x100 enteros se crea en la pila ($\text{int } m[100][100]$) como si se crea en memoria dinámica ($\text{int } **m$), el acceso al elemento situado en la posición (20, 20) se realiza de la misma manera: $\text{cout} \ll m[20][20]$.

| | Si no es necesario respetar el orden de los elementos de un vector dinámico, entonces el borrado de un dato en una posición arbitraria tiene el mismo orden de complejidad que el borrado en una lista doblemente enlazada.

| | Invertir el orden de los elementos de una lista simplemente enlazada requiere tiempo $O(n^2)$ (e.g. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \Rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$).

| | El recorrido de un árbol binario (inorden, preorden o postorden) requiere una pila para su implementación de forma que pueda volverse hacia atrás, es decir al nodo ancestro tras visitar los descendientes.

| | Todas las estructuras de datos de la familia de los árboles sirven para realizar búsquedas eficientes, con distintos

grados de eficiencia y diversos ámbitos de aplicación.

| | El factor máximo de carga recomendado (0.7) es sólo para las tablas de dispersión cerrada, donde las colisiones tienen un gran impacto en la eficiencia.

| | El principal problema de las mallas regulares es su incapacidad para adaptarse a conjuntos de datos espaciales que no se distribuyan de manera homogénea (i. e. con zonas con mucha densidad frente a otras vacías).

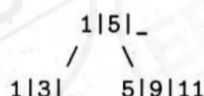
| | Como un árbol es un grafo conexo sin ciclos entonces un recorrido en anchura (BFS) empezando en la raíz es un recorrido del árbol por niveles.

| | Si en un fichero indexado el índice en memoria se desactualiza por algún problema en la aplicación, la operación de borrado puede realizar un borrado de un registro incorrecto.

| | La inserción de una clave en un árbol B puede implicar duplicaciones de nodos en todos los niveles del árbol.

Pregunta 2:

Dado el siguiente árbol B de orden 3, indicar la configuración del árbol tras la inserción/borrado de cada una de las siguientes claves: I(10), B(3), B(9), B(11), B(5)



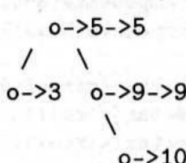
Pregunta 3:

Implementar las siguientes operaciones de estructuras de datos, incluyendo todo el código necesario (no asumir llamadas a operaciones cuyo código no se detalle aquí).

a) La operación `void ListaEnlazada<T>::insertarFinal(VDinamico<T> &v)` que inserta todos los elementos del vector `v` al final de una lista simplemente enlazada.

b) La operación `void MultiABB::insertar(T &dato)` en un árbol binario de búsqueda que admite datos repetidos. Para soportar tal funcionalidad la clase nodo tiene la implementación indicada a continuación, que permite guardar todas las datos asociados a la misma clave. A la hora de comparar un dato con los contenidos en el nodo, usar cualquiera de la lista (e. g. el primero del todos).

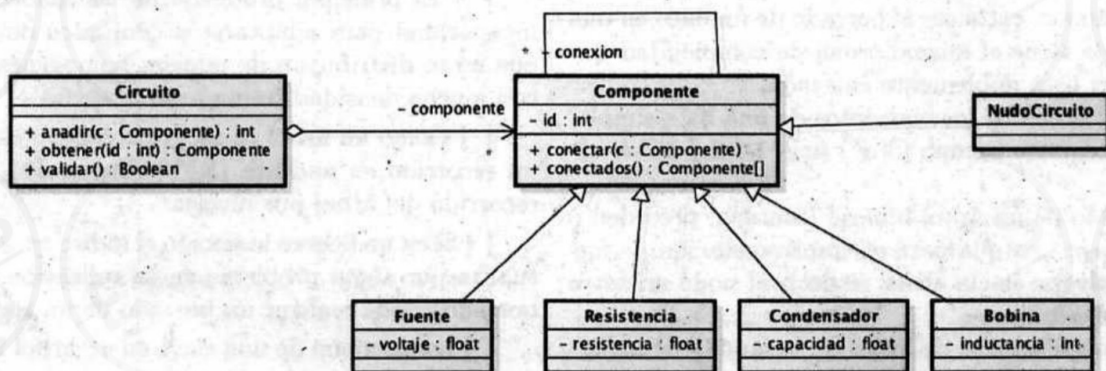
```
template<class T>
class Nodo {
public:
    Nodo<T> *izq, *der;
    ListaEnlazada<T> datos;
    Nodo(T &ele): izq(0), der(0), datos()
    { datos.insertarFinal(ele); }
};
```



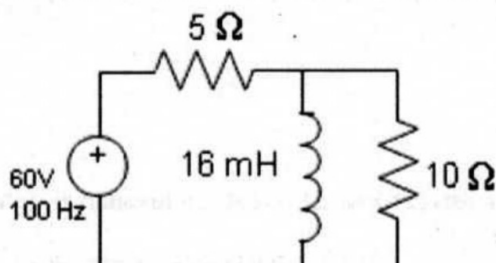
Pregunta 4:

Una aplicación de diseño de circuitos eléctricos necesita una estructura de datos para organizar los distintos componentes y la topología del circuito. El diagrama UML adjunto describe las clases de objetos necesarias.

- *Fuente*, *Resistencia*, *Condensador* y *Bobina* representan los componentes que admite la aplicación en su versión inicial. Cada uno de estos tres componentes viene definido por sus características eléctricas (voltaje, intensidad, resistencia, capacidad, inductancia). Estas clases heredan de *Componente*, que contiene un identificador único por componente y asociaciones a los otros componentes a los que está conectado. La operación `conectar()` permite conectar un componente con otro. Esta conexión debe ser siempre bidireccional: al conectar el componente A con B automáticamente se conecta B con A. La operación `conectados()` devuelve la lista de componentes conectados. Existe un componente especial denominado *NudoCircuito* que sirve para ayudar a crear bifurcaciones (aunque no tiene ninguna propiedad eléctrica).
- *Circuito* es la clase principal que organiza los componentes del circuito. Dispone de una operación para añadir un componente al circuito (`anadir()`) devolviendo el id del mismo (debe generar un identificador único para cada componente). De forma similar para obtener un componente a partir de su id usaremos `obtener()`. La operación más importante es `validar()`, que comprueba que el circuito está cerrado. Para ello deben verificarse las siguientes propiedades, en cuyo caso se devuelve true:
 1. Todos los componentes tienen 2 o más conexiones (para evitar circuitos abiertos).
 2. Empezando en un componente y siguiendo las conexiones se pueden alcanzar todos los demás componentes (para evitar que haya varios circuitos separados).



Implementar las clases *Circuito*, *Componente*, *NudoCircuito* y *Fuente* (considerar las otras tres clases implementadas, eligiendo las estructuras de datos adecuadas e implementando los algoritmos descritos. El siguiente es un ejemplo de creación y validación del circuito mostrado en la figura (este código debería funcionar con la implementación que se pide):



```
Circuito circuito;
int idFuente = circuito.anadirComponente(Fuente(60));
int idRest5 = circuito.anadirComponente(Resistencia(5));
int idNudo1 = circuito.anadirComponente(Nudo());
int idRest10 = circuito.anadirComponente(Resistencia(10));
int idBobina = circuito.anadirComponente(Bobina(16));
int idNudo2 = circuito.anadirComponente(Nudo());

circuito.obtener(idFuente)->conectar(circuito.obtener(idRest5));
circuito.obtener(idRest5)->conectar(circuito.obtener(idNudo1));
circuito.obtener(idNudo1)->conectar(circuito.obtener(idBobina));
circuito.obtener(idNudo1)->conectar(circuito.obtener(idRest10));
circuito.obtener(idBobina)->conectar(circuito.obtener(idNudo2));
circuito.obtener(idRest10)->conectar(circuito.obtener(idNudo2));
circuito.obtener(idNudo2)->conectar(circuito.obtener(idFuente));

if (circuito.validar())
    cout << "Circuito correcto\n";
else
    cout << "Circuito no válido\n";
```

Nota final: recordad que un circuito es un grafo. Los algoritmos estudiados pueden servir para resolver el problema propuesto.